ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		Д.А. Кочин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

УПРАВЛЕНИЕ ПАМЯТЬЮ

по курсу: ОПЕРАЦИОННЫЕ СИСТЕМЫ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2024

Цель работы:

Знакомство с принципами организации виртуальной памяти.

Задание на лабораторную работу:

В данной работе необходимо реализовать фрагмент диспетчера памяти и часть функционала операционной системы, отвечающего за замещение страниц при возникновении ошибок отсутствия страниц. Для упрощения работы предполагается использование линейной инвертированной таблицы страниц, работу с которой необходимо реализовать в виде программы. Также для простоты предполагается, что в системе имеется один единственный процесс, поэтому идентификатор процесса в инвертированной таблице страниц не хранится. Входные данные представляют собой последовательность операций обращения к памяти, выходные данные - состояние инвертированной таблицы страниц после каждой операции обращения к памяти.

1. Вычислить номер варианта по списку в журнале и сохранить его в файл TASKID.txt в репозитории.
2. Написать программу на языке C++ в соответствии со следующей спецификацией.
    i. Входные данные:
        a. Аргумент командной строки (число): номер алгоритма замещения страниц, который должна использовать программа. Принимает значения 1 или 2, соответствующие двум алгоритмам замещения страниц, заданным по варианту.
        b. Перечень инструкций обращения к памяти, считываемый программой из стандартного потока ввода. На каждой строке не более одной инструкции. Инструкция состоит из двух чисел, разделенных пробелом, например: 0 1. Первое число обозначает тип операции доступа к памяти: 0 - чтение и 1 -

запись. Второе число является номером виртуальной страницы, к которой происходит обращение.

ii. Выходные данные:

a. Для каждой операции обращения к памяти, информация о которой поступила на вход программы, на выходе должна быть сгенерирована строка, содержащая содержимое инвертированной таблицы страниц в виде последовательности номеров виртуальных страниц, разделенных пробелом. Если какая-либо из записей в таблице страниц отсутствует (таблица страниц не заполнена до конца), вместо номера виртуальной страницы необходимо вывести символ #.

3. Весь код поместить в файле lab4.cpp. Код должен корректно компилироваться командой g++ lab4.cpp -o lab4 -std=c++11. Настоятельно рекомендуется использовать стандартную библиотеку STL. Полезными могут быть контейнеры list, vector, bitset и др.
4. Если в работе алгоритма замещения страниц используется бит R, то необходимо реализовать эмуляцию прерывания таймера. Для этого через каждые 5 операций обращения к памяти необходимо запускать обработчик данного прерывания. Значения битов R по прерыванию таймера сбрасываются.
5. Для алгоритмов, использующих счетчик (NFU, Aging): если несколько страниц имеют одинаковое значение счетчика, одна из них выбирается случайным образом. При повторной загрузке страницы в память ее счетчик обнуляется. В алгоритме старения счетчик имеет размер 1 байт. В алгоритме NFU счетчик имеет размер не меньше 4 байт.
6. Во всех алгоритмах, использующих датчик случайных чисел (Random, NRU, NFU, Aging, ...), разрешается использовать **только** функцию int uniform_rnd(int a, int b), объявленную в файле lab4.h. Данная функция генерирует случайное целое число с равномерным распределением из

диапазона [a, b]. Использование других функций для работы со случайными числами запрещено!

7. В качестве системного времени в алгоритме рабочего набора следует использовать количество инструкций доступа к памяти, обработанных с момента запуска программы.
8. После успешного прохождения локальных тестов необходимо загрузить код в репозиторий на гитхабе.
9. Сделать выводы об эффективности реализованных алгоритмов замещения страниц. Сравнить количество ошибок отсутствия страниц, генерируемых на тестовых данных при использовании каждого алгоритма.
10. Подготовить отчет о выполнении лабораторной работы и загрузить его под именем report.pdf в репозиторий. В случае использования системы компьютерной верстки LaTeX также загрузить исходный файл report.tex.

Номер варианта	Количество страничных блоков	Алгоритм 1	Алгоритм 2
18	10	Random	NRU

Описание используемых алгоритмов замещения страниц:

Этот код реализует два алгоритма замещения страниц: случайный (Random Replacement, RND) и алгоритм наименее недавно используемой (Not Recently Used, NRU).

Описание алгоритмов

1. **Алгоритм случайного замещения (RND)**: Алгоритм случайного замещения обновляет страницы без учета их использования, заменяя случайную страницу, если новый запрос не находится в буфере. Он добавляет страницы до заполнения буфера, а затем случайным образом заменяет существующие страницы. Такой алгоритм прост в реализации, но неэффективен для часто используемых страниц, так как не учитывает их значимость.

2. **Алгоритм NRU (Not Recently Used)**: Алгоритм NRU более сложный и основан на классификации страниц по частоте и характеру использования. NRU разделяет страницы на 4 класса по двум признакам:

- **isUsedFlag**: индикатор, использовалась ли страница в последнем цикле (перезагружается каждые 5 итераций).
- **isModifyedFlag**: индикатор, была ли страница модифицирована.

В случае необходимости замены страницы выбирается случайная страница из самой низкой доступной категории:

- **Класс 0**: не использовалась и не модифицировалась.
- **Класс 1**: не использовалась, но была модифицирована.
- **Класс 2**: использовалась и модифицировалась.
- **Класс 3**: использовалась, но не модифицировалась.

Этот подход повышает вероятность сохранения часто используемых страниц в буфере, минимизируя ошибки страницы.

Результат выполнения работы:

```
parallels@ubuntu-linux-22-04-02-desktop:~/lab5/os-task4-Oblachko-Js$ g++ lab4.cp
p -o lab4 -std=c++11
parallels@ubuntu-linux-22-04-02-desktop:~/lab5/os-task4-Oblachko-Js$ ./tests.sh
Downloading shunit2-2.1.8 ...
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 52458    0 52458    0     0   104k      0 --:--:-- --:--:-- --:--:--  103k
test_TASKID
TASKID is 18
test_build
test_algorithm1
test_algorithm2

Ran 4 tests.

OK
parallels@ubuntu-linux-22-04-02-desktop:~/lab5/os-task4-Oblachko-Js$
```

Исходный код программы с комментариями:

```
#include <vector>
#include <iostream>
#include <string>
#include <sstream>

#include "lab4.h"

using namespace std;

#define PAGES_SIZE (10) // Определяем максимальный размер буфера страниц
static int counterOfIteration = 0; // Счетчик итераций для алгоритма NRU
int numErrorsOperation = 0; // Счетчик ошибок страницы

vector<int> bufferRndAlgo; // Буфер страниц для случайного алгоритма

// Функция вывода текущего состояния буфера случайного алгоритма
void outRndBuffer() {
    string str;
    for (int i = 0; i < PAGES_SIZE; ++i) {
        if (i < bufferRndAlgo.size())
            str.append(to_string(bufferRndAlgo[i]));
        else
            str.append("#");
        str.append(" ");
    }
    str.pop_back();
    cout << str << endl;
}
```

```
// Проверяет, содержится ли страница в буфере случайного алгоритма
bool checkRndBufContainPage(int page) {
    for (const auto &n : bufferRndAlgo) {
        if (n == page)
            return true;
    }
    return false;
}

// Выполняет алгоритм случайного замещения страниц
void rndAlgoExecute(){
    while (true) {
        string userLine;
        int operationNum = 0;
        int pageNum = 0;
        getline(cin, userLine); // Чтение строки от пользователя
        if (userLine.empty()) // Завершение при пустой строке
            return;

        if (userLine == "numErrorsOperation") // Команда вывода ошибок
страницы
        {
            cout << numErrorsOperation;
            return;
        }

        istringstream istream{ userLine };
        if (!(istream >> operationNum) || !(istream >> pageNum) || istream >>
userLine)
```

```
                continue; // Пропуск строки при неверном формате

            if (!checkRndBufContainPage(pageNum)) { // Если страницы нет в буфере
                numErrorsOperation++;
                if (bufferRndAlgo.size() < PAGES_SIZE) { // Добавление в буфер при наличии места
                    bufferRndAlgo.push_back(pageNum);
                }
                else {
                    int rndPageId = uniform_rnd(0, bufferRndAlgo.size() - 1); // Замена случайной страницы
                    bufferRndAlgo[rndPageId] = pageNum;
                }
            }
            outRndBuffer(); // Вывод буфера
        }
    }

    // Структура для страницы с данными алгоритма NRU
    struct NRUPageStruct {
        int num; // Номер страницы
        bool isUsedFlag; // Флаг использования
        bool isModifyedFlag; // Флаг модификации
    };

    vector<NRUPageStruct> bufferNruAlgo; // Буфер страниц для алгоритма NRU

    // Сброс флага использования для всех страниц
```

```
void checkTimer() {
    for (auto &virtualPage : bufferNruAlgo) {
        virtualPage.isUsedFlag = 0;
    }
    counterOfIteration = 0;
}

// Проверяет, содержится ли страница в буфере NRU
bool checkNruBufContainPage(int page) {
    for (auto &row : bufferNruAlgo) {
        if (row.num == page) {
            row.isUsedFlag = true;
            return true;
        }
    }
    return false;
}

// Процесс замещения страниц по алгоритму NRU
void nruProcess(int operation, int page) {
    if (counterOfIteration == 5)
        checkTimer(); // Сброс флагов каждые 5 итераций
    counterOfIteration++;

    if(checkNruBufContainPage(page))
        return;

    numErrorsOperation++;
```

```
bool isModifyOperation = operation == 1; // Установка флага модификации
int pageId = 0;

if (bufferNruAlgo.size() == PAGES_SIZE) { // Если буфер заполнен
    vector<int> classIdVecMode0, classIdVecMode1, classIdVecMode2, classIdVVecMode3;

    for (int i = 0; i < bufferNruAlgo.size(); ++i) { // Классификация страниц
        if (!bufferNruAlgo[i].isUsedFlag && !bufferNruAlgo[i].isModifyedFlag) {
            classIdVecMode0.push_back(i);
        }
        else if (!bufferNruAlgo[i].isUsedFlag && bufferNruAlgo[i].isModifyedFlag) {
            classIdVecMode1.push_back(i);
        }
        else if (bufferNruAlgo[i].isUsedFlag && bufferNruAlgo[i].isModifyedFlag) {
            classIdVecMode2.push_back(i);
        }
        else if (bufferNruAlgo[i].isUsedFlag && bufferNruAlgo[i].isModifyedFlag) {
            classIdVVecMode3.push_back(i);
        }
    }

    // Замена страницы на основании приоритета классов
    if (!classIdVecMode0.empty())
```

```
                pageId = classIdVecMode0[uniform_rnd(0, classIdVecMode0.size() -
1)];
            else if (!classIdVecMode1.empty())
                pageId = classIdVecMode1[uniform_rnd(0, classIdVecMode1.size() -
1)];
            else if (!classIdVecMode2.empty())
                pageId = classIdVecMode2[uniform_rnd(0, classIdVecMode2.size() -
1)];
            else if (!classIdVVecMode3.empty())
                pageId = classIdVVecMode3[uniform_rnd(0,
classIdVVecMode3.size() - 1)];

            bufferNruAlgo[pageId] = {page, true, isModifyOperation}; // Замена
страницы
        }
        else
            bufferNruAlgo.push_back({page, true, isModifyOperation}); //
Добавление новой страницы
    }

    // Вывод состояния буфера NRU
    void outNruBuffer() {
        string str;
        for (int i = 0; i < PAGES_SIZE; ++i) {
            if (i < bufferNruAlgo.size())
                str.append(to_string(bufferNruAlgo[i].num));
            else
                str.append("#");
            str.append(" ");
        }
```

```
        str.pop_back();
        cout << str << endl;
    }

    // Выполнение алгоритма NRU
    void nruAlgoExecute() {
        string userLine;
        int operationNum = 0;
        int pageNum = 0;

        while (true) {
            getline(cin, userLine); // Чтение строки от пользователя
            if (userLine.empty()) {
                return;
            }

            if (userLine == "numErrorsOperation")
            {
                cout << numErrorsOperation;
                return;
            }

            istringstream istream{ userLine };
            if (!(istream >> operationNum) || !(istream >> pageNum) || istream >>
userLine) {
                continue; // Пропуск строки при неверном формате
            }
            nruProcess(operationNum, pageNum); // Выполнение замены
страницы
            outNruBuffer(); // Вывод буфера
```

```
    }
}

int main(int argc, char *argv[]) {
    int argoNumber = atoi(argv[1]); // Определение типа алгоритма

    switch (argoNumber) {
    case 1:
        rndAlgoExecute(); // Запуск алгоритма RND
        break;
    case 2:
        nruAlgoExecute(); // Запуск алгоритма NRU
        break;
    default:
        break;
    }

    return 0;
}
```

Вывод:

Были изучены и реализованы два алгоритма замещения страниц в памяти: алгоритм случайного замещения (Random Replacement, RND) и алгоритм наименее недавно использованных страниц (Not Recently Used, NRU).

Алгоритм случайного замещения RND обеспечивает простую и быструю замену страниц, случайным образом выбирая страницы для удаления при переполнении буфера. Данный алгоритм не учитывает частоту использования страниц, что делает его менее эффективным в условиях частого повторного обращения к одним и тем же данным.

Алгоритм NRU, напротив, реализует замену на основе классификации страниц по уровням важности, что позволяет минимизировать количество ошибок страниц. NRU делит страницы на четыре класса, исходя из признаков использования и модификации страниц. При необходимости замены страницы алгоритм выбирает страницу из самой низкой категории, что позволяет сохранить страницы с более высокой вероятностью повторного обращения. Этот подход делает NRU более эффективным для приложений, где часто используются одни и те же данные.